

Curso de Go

Aula 7 - Maps e Structs

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_** [3]
BRAZUCA





Nessa Aula nos Vamos Ver

- Entender como usar maps como dicionários
- Trabalhar com leitura, escrita e remoção segura
- Aprender structs como base de modelagem
- Entender composição como substituto da herança
- Parte Prática



O que são Maps em Go

- **Maps são estruturas chave → valor (key-value), equivalentes a dicionários.**
- **Características:**
 - Não possuem ordem garantida
 - Acesso rápido $O(1)$ em média
 - Chaves devem ser comparáveis (string, int, etc.)
- **Exemplo conceitual:**
 - nome → "Antonio"
 - idade → 42



Criação de Maps

- **Usando make (mais comum):**

Exemplo de Código:

```
usuarios := make(map[string]int)
```

- **Inicialização Direta:**

Exemplo de Código:

```
usuarios := map[string]int{  
    "Antonio": 42,  
    "Maria": 30,  
}
```

- **Explicação:**

- make aloca e inicializa o map
- map[...]{} já cria com valores iniciais



Escrita em Maps

- **Inserção e atualização são feitas da mesma forma:**

Exemplo de Código:

```
usuarios := make(map[string]int)
```

```
usuarios["Antonio"] = 42
```

```
usuarios["Maria"] = 30
```

```
// Atualização
```

```
usuarios["Antonio"] = 43
```

- **Explicação:**
 - **Se a chave não existe, cria**
 - **Se existe, sobrescreve**



Leitura de Dados em Maps

- **Acesso direto:**

Exemplo de Código:

```
idade := usuarios["Antonio"]  
fmt.Println(idade)
```

- **Problema:**

- **Se a chave não existir, retorna valor zero (0, "", false...)**
- **Isso pode gerar bugs silenciosos.**



Verificação de Existência (forma segura)

- **Forma correta de leitura:**

Exemplo de Código:

```
idade, ok := usuarios["Joao"]

if ok {
    fmt.Println("Idade:", idade)
} else {
    fmt.Println("Usuário não encontrado")
}
```

- **Explicação:**

- **ok é booleano**
- **evita confundir valor zero com valor inexistente**



Exclusão de Chaves

- **Forma correta de leitura:**

Exemplo de Código:

```
delete (usuarios, "Maria")
```

- **Características:**

- **Seguro mesmo se a chave não existir**
- **Não gera erro**

- **Boa prática: sempre tratar a existência antes de operar dados críticos.**



Iteração em Maps

- **Percorrendo um map:**

Exemplo de Código:

```
for nome, idade := range usuarios {  
    fmt.Println(nome, idade)  
}
```

- **Observação importante:**
 - **A ordem não é garantida**
 - **Cada execução pode gerar ordem diferente**



Structs: conceito e por que usar

- **Structs são tipos compostos que permitem agrupar dados relacionados em uma única estrutura.**
- **Exemplos**

Sem structs

Exemplo de Código:

```
nome := "Antonio"  
idade := 42  
cidade := "Rio"
```

Com structs

Exemplo de Código:

```
type Usuario struct {  
    Nome    string  
    Idade   int  
    Cidade  string  
}
```



Vantagens do Uso de Structs

- **Organização de dados**
- **Melhor legibilidade**
- **Facilita evolução do sistema**
- **Base para modelagem de domínio**

Exemplo de Código:

```
u := Usuario{
  Nome: "Antonio",
  Idade: 42,
  Cidade: "Rio",
}
```



Inicialização e formas de criação

- **Nomeada (mais segura):**

Exemplo de Código:

```
u := Usuario{
  Nome: "Antonio",
  Idade: 42,
}
```

- **Com zero value:**

Exemplo de Código:

```
u.Nome = "Antonio"
```

- **Explicação importante:**

- **Campos não inicializados recebem zero value**
- **string** → ""
- **int** → 0

- **Posicional (evitar em produção):**

Exemplo de Código:

```
u := Usuario{"Antonio", 42,
  "Rio"}
```

- **Problema:**

- **Dependência da ordem dos campos**
- **Facilmente quebra ao evoluir a struct**



Structs, memória e cópia

- **Structs são valores (value types)**
- **Isso significa que são copiadas**

Exemplo de Código:

```
u1 := Usuario{Nome: "Antonio"}  
u2 := u1  
  
u2.Nome = "Maria"  
  
fmt.Println(u1.Nome) // Antonio
```




Structs, memória e cópia

- **Cada variável tem sua própria cópia.**
- **Para evitar cópia, usar ponteiro**

Exemplo de Código:

```
u1 := Usuario{Nome: "Antonio"}  
u2 := &u1
```

```
u2.Nome = "Maria"
```

```
fmt.Println(u1.Nome) // Maria
```

- **Quando usar ponteiros:**
 - **Structs grandes**
 - **Performance**
 - **Mutabilidade compartilhada**



Structs anônimos e uso prático

- **Structs anônimos não têm nome de tipo.**
- **Casos de uso:**
 - **Dados temporários**
 - **Respostas de API**
 - **Testes rápidos**
 - **Agrupamentos locais**

Exemplo de Código:

```
func getResumo() struct {  
    Total int  
    Ativo bool  
} {  
    return struct {  
        Total int  
        Ativo bool  
    }{Total: 10, Ativo: true}  
}
```



Composição: modelando sistemas reais

- **Composição é a forma de reutilizar código em Go.**

Exemplo de Código 1:

```
u := Usuario{
  Nome: "Antonio",
  Endereco: Endereco{
    Rua: "Rua A",
    Cidade: "Rio",
  },
}
```

Exemplo de Código Acesso Direto:

```
fmt.Println(u.Cidade)
}
```

- **Explicação**
 - **Endereco é "embutido"**
 - **Campos são promovidos**
 - **Não precisa de herança**



Composição vs Herança (mentalidade Go)

- **Herança (em outras linguagens):**
 - **Hierarquia rígida**
 - **Forte acoplamento**
- **Go usa composição:**
 - **Estruturas pequenas e reutilizáveis**
 - **Baixo acoplamento**
 - **Mais flexível**

Exemplo de Código 1:

```
type Logger struct{}

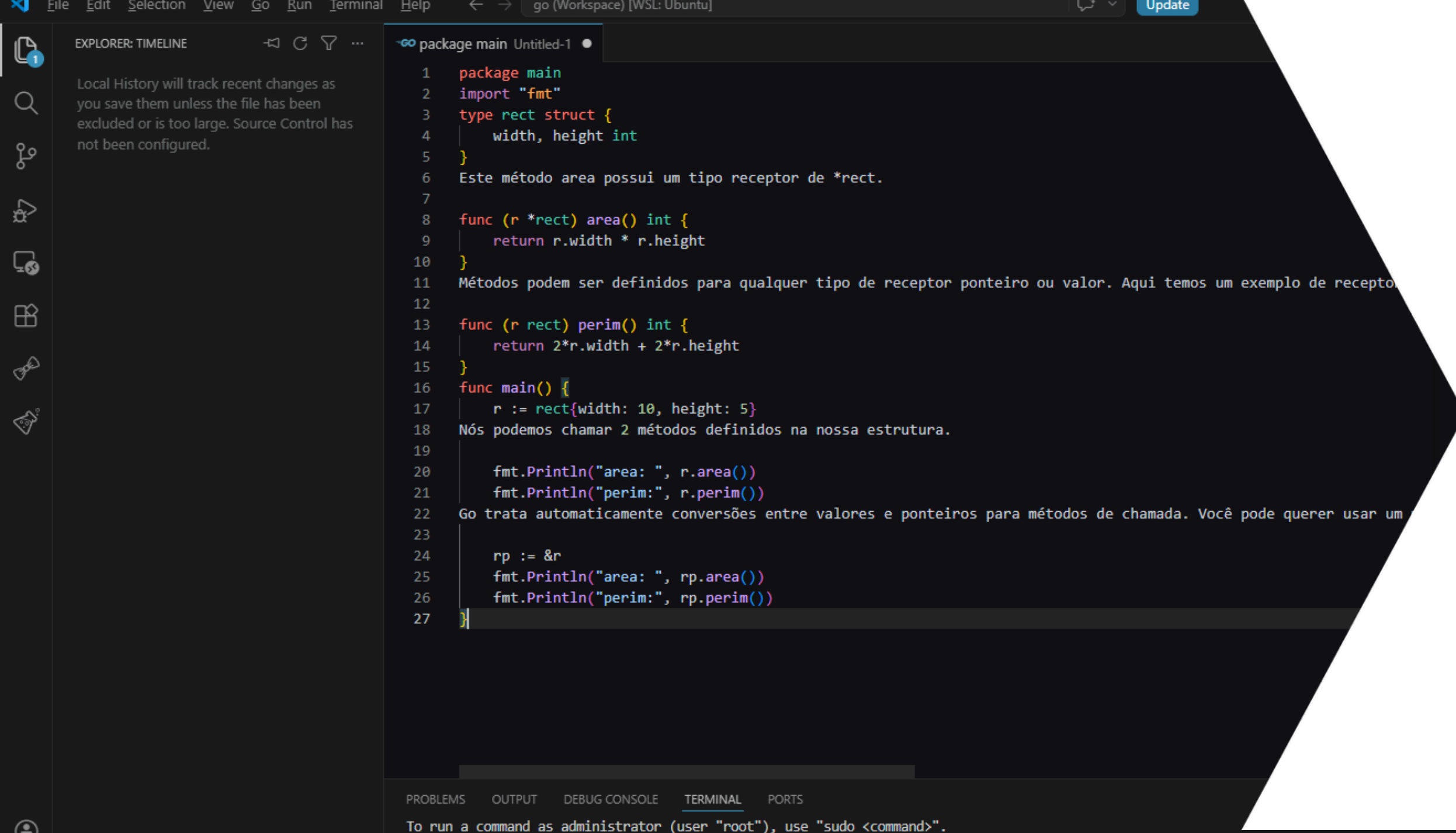
func (l Logger) Log(msg string) {
    fmt.Println("LOG:", msg)
}
```

```
type Servico struct {
    Logger
}
```

Uso:

```
s := Servico{}
s.Log("Executando serviço")
```

- **Insight importante:**
 - **Você “injeta comportamento” via composição**
 - **Isso substitui herança + mixins**



The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' package. The code defines a struct 'rect' with 'width' and 'height' fields of type 'int'. It then defines two methods: 'area()' which returns the product of width and height, and 'perim()' which returns the perimeter (2*width + 2*height). In the 'main()' function, a 'rect' struct is created with width 10 and height 5. It prints the area and perimeter using 'fmt.Println'. A pointer 'rp' is created to the 'rect' struct, and the same area and perimeter calculations are performed using the pointer. The IDE interface includes a sidebar with 'EXPLORER: TIMELINE' and a bottom status bar with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Maps e Structs